

- QualiTest — L2 Technical Deep-Dive (Part 2)
- AWS + SRE Practices + Scenario-Based Questions
 - SECTION 4: AWS INFRASTRUCTURE (4+ Years Required)
 - Q13. How do you design a production AWS architecture for a web application?
 - Q14. Explain AWS IAM best practices you implement.
 - Q15. How do you implement auto-scaling on AWS?
 - SECTION 5: SRE & OPERATIONAL EXCELLENCE
 - Q16. What SRE practices do you implement for system reliability?
 - Q17. How do you set up monitoring and alerting for infrastructure?
 - Q18. Walk through a deployment gone wrong. How do you handle rollbacks?
 - SECTION 6: SCENARIO-BASED QUESTIONS (L2 Focus)
 - Q19. A developer says "my build is failing in CI but works on my machine." How do you debug?
 - Q20. How would you migrate a company from Jenkins to GitLab CI?
 - Q21. Your production ECS service is throwing OOM (Out of Memory) errors. Walk through diagnosis and fix.
 - Q22. Design a disaster recovery strategy for a production application.
 - Q23. How do you handle log management at scale?
 - SECTION 7: QUICK-FIRE QUESTIONS
 - Q24. What's the difference between a container and a VM?
 - Q25. Explain Docker multi-stage builds.
 - Q26. What happens when you type `curl https://example.com`?
 - Q27. How do you secure a Docker container?
 - FILES IN THIS SERIES

QualiTest — L2 Technical Deep-Dive (Part 2)

AWS + SRE Practices + Scenario-Based Questions

Role: Senior DevOps/SRE Engineer | **Round:** L2 Technical (1 Hour)

SECTION 4: AWS INFRASTRUCTURE

(4+ Years Required)

Q13. How do you design a production AWS architecture for a web application?

Answer:

```
Route53 (DNS)
|
CloudFront (CDN + WAF)
|
ALB (Application Load Balancer) – public subnets (multi-AZ)
|
ECS Fargate / EKS – private subnets (multi-AZ)
|
Aurora PostgreSQL – isolated subnets (multi-AZ, read replicas)
|
ElastiCache Redis – session store, caching
|
S3 – static assets, logs, backups
```

Networking:

```
# VPC: 10.0.0.0/16
# Public subnets: 10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24 (ALB, NAT GW)
# Private subnets: 10.0.10.0/24, 10.0.11.0/24, 10.0.12.0/24 (App tier)
# Isolated subnets: 10.0.20.0/24, 10.0.21.0/24, 10.0.22.0/24 (DB – no
internet)
```

Key design decisions:

- **Multi-AZ everything** — ALB, ECS tasks, Aurora, ElastiCache all span 3 AZs
- **Private subnets for compute** — No direct internet access; outbound via NAT Gateway
- **VPC endpoints** for AWS services (S3, ECR, Secrets Manager) — traffic stays on AWS backbone

- **Security groups** as micro-firewalls — ALB SG allows 443 from internet; App SG allows 8080 from ALB SG only; DB SG allows 5432 from App SG only
 - **Encryption everywhere** — KMS CMKs, TLS 1.2+ enforced, S3 SSE-KMS
-

Q14. Explain AWS IAM best practices you implement.

Answer:

Core principles:

1. **No root account usage** — MFA on root, locked in a safe, only for break-glass scenarios
2. **SSO via Identity Center** — No IAM users with passwords; federate via Okta/Azure AD
3. **Least privilege roles:**

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::app-bucket/uploads/*",
  "Condition": {
    "StringEquals": {"aws:RequestedRegion": "us-east-1"},
    "IpAddress": {"aws:SourceIp": "10.0.0.0/8"}
  }
}
```

4. **Permission boundaries** — Max permissions any role can have, even if policy allows more
5. **SCPs at Org level:**

```
{
  "Effect": "Deny",
  "Action": ["ec2:RunInstances"],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {"ec2:Region": ["us-east-1", "ap-south-1"]}
  }
}
```

6. **IAM Access Analyzer** — Continuously flags overly permissive policies and unused permissions
 7. **Short-lived credentials** — STS temporary tokens, never long-lived access keys
 8. **Separate accounts per environment** — AWS Organizations with dev/staging/prod accounts
-

Q15. How do you implement auto-scaling on AWS?

Answer:

ECS Fargate auto-scaling (what I typically implement):

```
# Target tracking — maintain 60% average CPU
resource "aws_appautoscaling_target" "ecs" {
  service_namespace = "ecs"
  resource_id        = "service/${var.cluster_name}/${var.service_name}"
  scalable_dimension = "ecs:service:DesiredCount"
  min_capacity       = 2
  max_capacity       = 20
}

resource "aws_appautoscaling_policy" "cpu" {
  name                = "cpu-scaling"
  service_namespace   = "ecs"
  resource_id         = aws_appautoscaling_target.ecs.resource_id
  scalable_dimension  = "ecs:service:DesiredCount"
  policy_type         = "TargetTrackingScaling"

  target_tracking_scaling_policy_configuration {
    target_value          = 60.0
    predefined_metric_specification {
      predefined_metric_type = "ECSServiceAverageCPUUtilization"
    }
    scale_in_cooldown = 300
    scale_out_cooldown = 60    # Scale out fast, scale in slow
  }
}
```

Multi-metric scaling strategy:

- **CPU tracking** at 60% — handles compute-bound load
- **Custom metric** on SQS queue depth — scale workers based on backlog, not CPU

- **Scheduled scaling** — pre-scale before known peak hours (e.g., 9 AM business start)
 - **Scale-out cooldown: 60s** — respond fast to spikes
 - **Scale-in cooldown: 300s** — avoid flapping (premature scale-down causing re-scale)
-

SECTION 5: SRE & OPERATIONAL EXCELLENCE

Q16. What SRE practices do you implement for system reliability?

Answer:

SLIs, SLOs, and Error Budgets:

SLI (Service Level Indicator):

- Availability: $(\text{successful requests} / \text{total requests}) \times 100$
- Latency: P99 response time
- Error rate: $5xx \text{ responses} / \text{total responses}$

SLO (Service Level Objective):

- Availability: 99.95% ($\approx 22 \text{ min downtime/month}$)
- Latency P99: $< 500\text{ms}$
- Error rate: $< 0.1\%$

Error Budget:

- $100\% - 99.95\% = 0.05\% = \sim 22 \text{ min/month}$
- If error budget is consumed → freeze feature releases, focus on reliability
- If error budget is healthy → deploy faster, take more risks

Incident management:

1. **Detection:** CloudWatch alarms → PagerDuty → on-call engineer (within 5 min)
2. **Response:** Acknowledge → Assess severity → Start incident channel (Slack)
3. **Mitigation:** Rollback/scale/failover first, investigate root cause second
4. **Resolution:** Apply permanent fix through CI/CD

5. **Post-mortem:** Blameless review within 48 hours, action items tracked to completion

Blameless post-mortem template:

Incident: API latency spike – 2026-05-12

Timeline

- 14:02 – Alert fired: API P99 > 2s
- 14:07 – On-call acknowledged, started investigation
- 14:15 – Root cause: DB connection pool exhausted
- 14:20 – Mitigation: Increased pool size via config change
- 14:25 – Verified: Latency back to normal

Root Cause

Connection pool max was 10, under load hit limit. New connections queued.

Action Items

- [] Increase pool size to 50 in all environments (owner: @dev, due: May 14)
- [] Add CloudWatch metric for active DB connections (owner: @sre, due: May 15)
- [] Load test with 2x expected traffic monthly (owner: @qa, due: May 20)

Q17. How do you set up monitoring and alerting for infrastructure?

Answer:

Monitoring stack I implement:

```
Infrastructure Metrics → CloudWatch / Prometheus
Application Logs       → CloudWatch Logs / ELK / Loki
Distributed Traces     → X-Ray / Jaeger
Dashboards             → Grafana / CloudWatch Dashboards
Alerting               → CloudWatch Alarms → SNS → PagerDuty
Synthetic Monitoring   → CloudWatch Synthetics (canary URLs)
```

Alert design principles — avoid alert fatigue:

Severity	Criteria	Response	Example
P1	Customer-facing, revenue-impacting	PagerDuty page, immediate	API down, error rate > 5%

Severity	Criteria	Response	Example
P2	Degraded but functional	Slack alert, 30 min response	Latency P99 > 1s, one AZ unhealthy
P3	Potential future issue	Ticket, next business day	Disk > 70%, certificate expires in 14 days
P4	Informational	Dashboard only	Deployment completed, scaling event

Key metrics I always monitor:

```
# Application (RED method)
Rate:    requests/sec per endpoint
Errors:  5xx/sec, 4xx/sec
Duration: P50, P95, P99 latency

# Infrastructure (USE method)
Utilization: CPU%, memory%, disk%, network bandwidth
Saturation:  Queue depth, thread pool usage, connection pool
Errors:      Hardware errors, packet drops, OOM kills

# Business
Orders/minute, payment success rate, user signups/hour
```

Q18. Walk through a deployment gone wrong. How do you handle rollbacks?

Answer:

Scenario: Deployed v2.5 of the API service. After 10 minutes, error rate climbs from 0.1% to 8%.

Immediate response (first 5 minutes):

```
# ArgoCD – instant rollback to previous Git commit
argocd app history payment-service
# ID    DATE          REVISION
# 5     2026-05-12    abc123 (v2.5 – bad)
# 4     2026-05-10    def456 (v2.4 – good)

argocdapp rollback payment-service 4
```

```
# ArgoCD syncs cluster back to v2.4 in ~30 seconds

# OR if using GitLab:
git revert abc123
git push origin main
# ArgoCD auto-syncs to the reverted state
```

For Lambda/ECS (non-K8s):

```
# Lambda – shift alias back
aws lambda update-alias \
  --function-name payment-api \
  --name prod \
  --function-version 42 # Previous known-good version

# ECS – update service to previous task definition
aws ecs update-service \
  --cluster prod \
  --service payment-api \
  --task-definition payment-api:41 # Previous revision
```

Prevention for next time:

- **Canary deployments:** Route 5% traffic to new version, monitor for 10 minutes, then promote
- **Automated rollback:** ArgoCD Analysis template — if error rate > 1%, auto-rollback
- **Feature flags:** Deploy code but keep feature disabled. Enable via flag after validation
- **Smoke tests:** Post-deploy automated tests that hit critical endpoints

SECTION 6: SCENARIO-BASED QUESTIONS (L2 Focus)

Q19. A developer says "my build is failing in CI but works on my machine." How do you debug?

Answer:

Systematic approach:

1. Environment differences (most common)
 - Check: Node/Python/Java version in CI vs local
 - Fix: Pin versions in Dockerfile or CI config
 - Prevention: Developers run in Docker containers matching CI image
2. Dependencies
 - Check: Is package-lock.json/requirements.txt committed?
 - Fix: Run ``npm ci`` (not ``npm install``) in CI for deterministic installs
 - Check: Private registry accessible from CI runners?
3. File system / permissions
 - Check: Case sensitivity – macOS is case-insensitive, Linux CI is case-sensitive
 - Check: File permissions – scripts need ``chmod +x``
 - Check: Line endings – Windows CRLF vs Linux LF
4. Environment variables
 - Check: Are required env vars set in CI?
 - Fix: Add to GitLab CI variables (masked for secrets)
5. Resource limits
 - Check: CI runner memory/CPU limits vs local machine
 - Fix: Increase runner resources or optimize build
6. Network
 - Check: Can CI runner reach external services (npm registry, APIs)?
 - Fix: Configure proxy settings or mirror registries in Artifactory

My standard practice: Every project has a **Makefile** or **docker-compose.dev.yml** that developers use locally — the **exact same image** used by CI. "Works on my machine" should never happen.

Q20. How would you migrate a company from Jenkins to GitLab CI?

Answer:

Phase 1 — Assessment (Week 1-2):

- Inventory all Jenkins jobs (builds, deployments, scheduled tasks)
- Map each Jenkinsfile stage to GitLab CI equivalent

- Identify Jenkins plugins in use and GitLab alternatives
- Document credentials/secrets stored in Jenkins

Phase 2 — Parallel run (Week 3-6):

- Create `.gitlab-ci.yml` for each project alongside existing Jenkinsfile
- Run both pipelines in parallel — compare results
- Migrate secrets to GitLab CI/CD variables

Phase 3 — Switchover (Week 7-8):

- Disable Jenkins jobs one project at a time
- Monitor GitLab pipelines for 1 week per project
- Update team documentation and training

Jenkins to GitLab mapping:

Jenkins	GitLab CI
Jenkinsfile (stages)	<code>.gitlab-ci.yml</code> (stages)
Plugins	Built-in features or <code>include</code> templates
Credentials	CI/CD Variables (masked, protected)
Shared libraries	CI <code>include</code> + project templates
Agents/nodes	GitLab Runners (shared or specific)
Multibranch pipeline	Auto-DevOps or <code>rules: if</code> per branch
Blue Ocean UI	Built-in pipeline visualization

Q21. Your production ECS service is throwing OOM (Out of Memory) errors. Walk through diagnosis and fix.

Answer:

```
# Step 1: Confirm OOM kills
aws ecs describe-services --services payment-api --cluster prod
# Check: runningCount vs desiredCount (tasks being killed and restarted)
```

```

# Check CloudWatch for ECS task-level metrics
# Metric: MemoryUtilization approaching 100% before task stops

# Step 2: Check task definition memory allocation
aws ecs describe-task-definition --task-definition payment-api:latest
# If hard limit = 512MB but app needs 800MB → OOM kill

# Step 3: Analyze application memory
# Enable container insights for per-container memory breakdown
# Check if memory grows over time (leak) or spikes suddenly (burst)

# Step 4: Fix options (in order of preference):

# Option A: Increase task memory (quick fix)
# In Terraform:
resource "aws_ecs_task_definition" "app" {
    memory = "1024" # Was 512, increase to 1024
    cpu    = "512"
}

# Option B: Fix the memory leak (proper fix)
# Profile the application (heap dump for Java, memory_profiler for Python)
# Common causes: unclosed DB connections, growing caches, event listener leaks

# Option C: Add swap (ECS supports this)
# Buys time while fixing the root cause

# Step 5: Add monitoring to prevent recurrence
resource "aws_cloudwatch_metric_alarm" "memory_high" {
    alarm_name        = "ecs-memory-high"
    comparison_operator = "GreaterThanThreshold"
    evaluation_periods = 2
    metric_name       = "MemoryUtilization"
    namespace         = "AWS/ECS"
    period            = 300
    threshold          = 80 # Alert at 80% before OOM at 100%
    statistic          = "Average"
    alarm_actions      = [aws_sns_topic.alerts.arn]
}

```

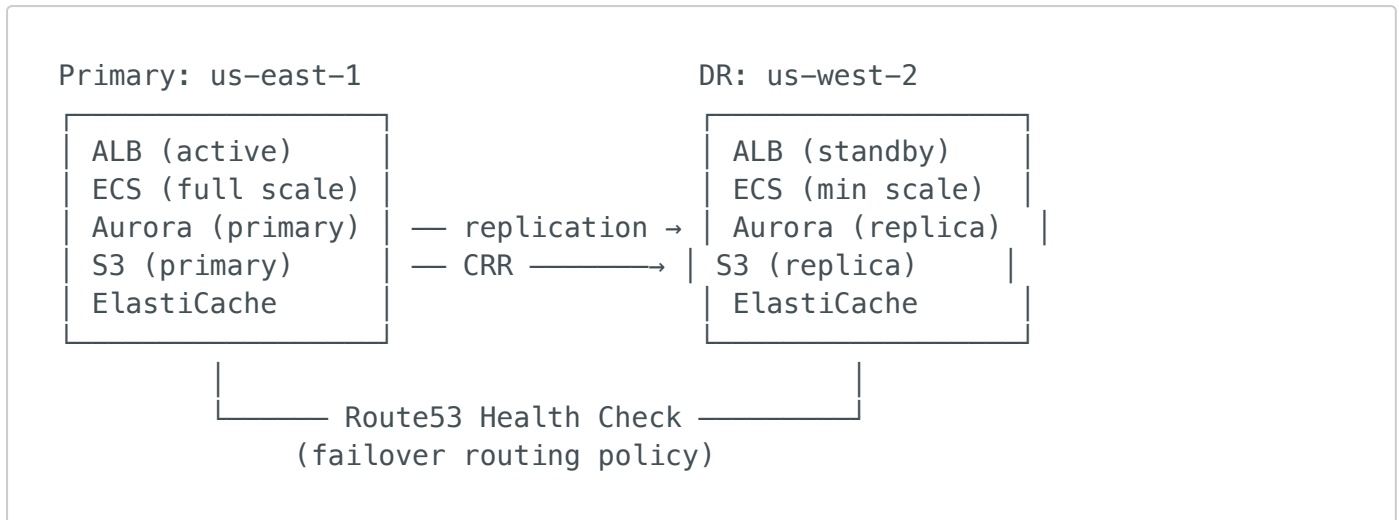
Q22. Design a disaster recovery strategy for a production application.

Answer:

DR tiers and trade-offs:

Strategy	RPO	RTO	Cost	Use Case
Backup & Restore	Hours	Hours	\$	Non-critical apps
Pilot Light	Minutes	30-60 min	\$\$	Important apps
Warm Standby	Seconds	Minutes	\$\$\$	Business-critical
Multi-site Active-Active	Zero	Zero	\$\$\$\$	Revenue-critical

My typical Warm Standby implementation:



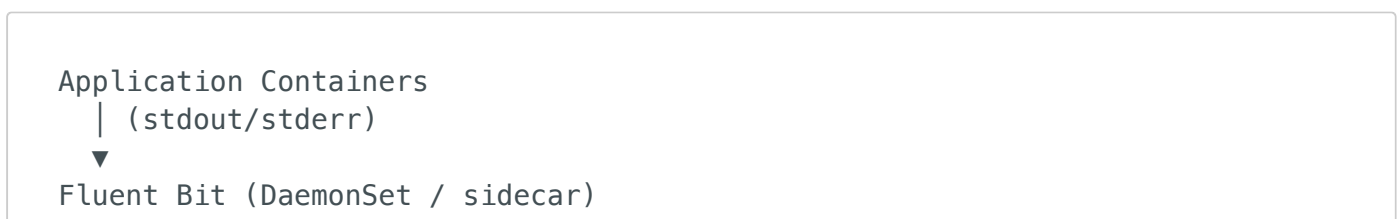
Failover process:

1. Route53 health check detects primary ALB unhealthy
2. DNS automatically routes to DR ALB (TTL: 60s)
3. ECS auto-scales from minimum (2 tasks) to full capacity (10 tasks)
4. Aurora replica promotes to primary (typically < 2 minutes)
5. Application connects to promoted Aurora instance

Testing: Quarterly DR drill — actually failover to DR region, run for 4 hours, failback.

Q23. How do you handle log management at scale?

Answer:



- CloudWatch Logs (operational – 30-day retention)
- S3 (archival – compressed, lifecycle to Glacier after 90 days)
- OpenSearch (search & analysis – 14-day hot storage)

Key practices:

```
# Structured logging – JSON, not plain text
import json, logging

logger = logging.getLogger()
logger.info(json.dumps({
    "level": "INFO",
    "message": "Order processed",
    "order_id": "ORD-12345",
    "customer_id": "CUST-789",
    "amount": 149.99,
    "duration_ms": 234,
    "trace_id": "abc-123-def",
    "timestamp": "2026-05-12T08:00:00Z"
}))
```

Why structured logging matters:

- CloudWatch Logs Insights can query JSON fields: `filter order_id = "ORD-12345"`
- Grafana/Kibana dashboards aggregate by any field
- Correlation via `trace_id` across microservices
- Machine-parseable for automated alerting on specific patterns

Cost optimization:

- **Log levels in CI variable** — production: WARN+ERROR only; staging: DEBUG
- **Sampling** — log 10% of successful requests, 100% of errors
- **Retention policies** — 7 days hot (OpenSearch), 30 days warm (CloudWatch), 1 year cold (S3 Glacier)
- **Subscription filters** — only forward ERROR logs to alerting pipeline

SECTION 7: QUICK-FIRE QUESTIONS

Q24. What's the difference between a container and a VM?

Aspect	VM	Container
Isolation	Full OS + hypervisor	Shared kernel, namespace isolation
Size	GBs (full OS image)	MBs (app + dependencies only)
Startup	Minutes	Seconds
Resource overhead	High (runs full OS)	Low (shares host kernel)
Use case	Strong isolation needed, legacy apps	Microservices, CI/CD, cloud-native

Q25. Explain Docker multi-stage builds.

```
# Stage 1: Build (large image with build tools)
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Production (small image, no build tools)
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
USER node
CMD ["node", "dist/server.js"]
```

Why: Final image is 50-80% smaller — no compiler, no dev dependencies, smaller attack surface.

Q26. What happens when you type `curl https://example.com`?

1. DNS resolution: resolve `example.com` → IP address (check `/etc/hosts` → local DNS cache → resolver)
2. TCP handshake: SYN → SYN-ACK → ACK (3-way handshake)
3. TLS handshake: Client Hello → Server Hello + Certificate → Key Exchange → Finished
4. HTTP request: GET / HTTP/1.1, Host: `example.com`
5. Server processes request and returns HTTP response
6. Client receives response body
7. Connection close (or keep-alive for reuse)

Q27. How do you secure a Docker container?

1. Use minimal base images (alpine, distroless)
2. Run as non-root user (USER 1000)
3. Scan images for CVEs (Trivy, Snyk)
4. Don't store secrets in images – inject at runtime
5. Read-only filesystem where possible
6. Drop all Linux capabilities, add back only needed ones
7. Set resource limits (memory, CPU)
8. Use multi-stage builds – no build tools in production image
9. Sign images and verify before deployment
10. Regularly rebuild to pick up base image patches

FILES IN THIS SERIES

File	Content
QualiTest_L2_Technical_Part1.md	CI/CD (GitLab, ArgoCD, Artifactory), Linux troubleshooting, Terraform
QualiTest_L2_Technical_Part2.md	AWS architecture, SRE practices, scenario questions, quick-fire

*Prepared for: QualiTest Senior DevOps/SRE — L2 Technical Interview Candidate:
Pushparaj Naik*